

LAPORAN PRAKTIKUM PEKAN 3

STRUKTUR DATA

Disusun Oleh:

Muhammad Fharel

2511531010

Dosen Pengampu:

Dr. Wahyudi S.T.M.T

Asisten Pratikum:

Muhammad Galid Avero



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Segala puji penulis panjatkan ke hadirat Allah SWT atas rahmat dan karunia-Nya sehingga laporan praktikum Struktur Data pada tanggal 8 April 2026 yang membahas tentang konsep dasar struktur data *Stack* beserta penerapannya menggunakan berbagai pendekatan. Selain itu, juga memahami bagaimana operasi dasar *Stack* seperti *push*, *pop*, dan *peek* digunakan dalam pengolahan data, termasuk dalam kasus nyata seperti evaluasi ekspresi *postfix*. Materi ini penting karena menjadi fondasi dalam memahami struktur data.

Ucapan terima kasih ditujukan kepada dosen pengampu, asisten praktikum, serta rekan-rekan yang telah membantu dalam proses pelaksanaan praktikum. Penulis menyadari bahwa penulisan laporan masih jauh dari kata sempurna, oleh karena itu kritik dan saran sangat diharapkan untuk penyempurnaan di kemudian hari. Semoga laporan ini memberikan manfaat dan menambah wawasan pembaca.

Padang, 18 April 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI	ii
BAB I PENDAHULUAN	1
1.1 Latar Belakang.....	1
1.2 Tujuan Praktikum	1
1.3 Manfaat Praktikum	1
BAB II PEMBAHASAN	3
2.1 Stack	3
2.2 Kode Pemograman	3
2.2.1 Class StackArray.....	3
2.2.2 Class StackArray Driver	4
2.2.3 Output StackArray Driver	5
2.2.4 Class Stack Postfix.....	6
2.2.5 Output Stack Postfix	7
2.2.6 Class Nilai Maksimum.....	7
2.2.7 Output Nilai Maksimum	8
2.2.8 Class Siswa Stack.....	8
2.2.9 Output Siswa Stack	10
2.2.10 Class Latihan Stack.....	10
2.2.11 Output Latihan Stack.....	11
BAB III PENUTUP	12
3.1 Kesimpulan.....	12
DAFTAR PUSTAKA	13

TUGAS PRAKTIKUM.....	14
----------------------	----

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam pemrograman, pengelolaan data secara terstruktur sangat penting untuk mempermudah proses pengolahan data. Salah satu struktur data yang sering digunakan adalah *Stack*. *Stack* merupakan struktur data yang bekerja dengan prinsip LIFO (*Last In First Out*), yaitu data yang terakhir masuk akan menjadi data pertama yang keluar.

Stack banyak digunakan dalam berbagai kasus, seperti evaluasi ekspresi matematika, *undo/redo* pada aplikasi, dan pengolahan data sementara. Dalam Java, *Stack* dapat diimplementasikan menggunakan *array*, *ArrayList*, maupun *class* bawaan seperti *Stack*. Pada praktikum ini, dilakukan beberapa implementasi *Stack* dengan berbagai pendekatan untuk memahami cara kerja dan penggunaannya dalam program.

1.2 Tujuan Praktikum

1. Memahami konsep dasar struktur data *Stack*.
2. Mengetahui prinsip kerja LIFO.
3. Mampu mengimplementasikan *Stack* menggunakan *array* dan Java *Collection*.
4. Memahami operasi dasar *Stack* (*push*, *pop*, *peek*).
5. Menerapkan *Stack* dalam kasus nyata seperti *postfix*.

1.3 Manfaat Praktikum

1. Memahami cara kerja struktur data *Stack* dengan prinsip LIFO (*Last In First Out*).
2. Meningkatkan kemampuan dalam mengelola data yang bersifat sementara (*temporary data*).

3. Melatih logika dalam proses pengambilan dan pengembalian data secara berurutan.
4. Menjadi dasar untuk memahami struktur data lain yang berkaitan, seperti *queue* dan *tree*.

BAB II

PEMBAHASAN

2.1 Stack

Stack adalah struktur data yang bekerja dengan prinsip *Last In First Out* (LIFO), yaitu data yang terakhir masuk akan keluar terlebih dahulu. Konsep ini mirip seperti tumpukan benda, di mana yang berada paling atas akan diambil lebih dulu. *Stack* banyak digunakan dalam pemrograman, seperti pada fitur *undo/redo* dan perhitungan ekspresi matematika.

Dalam Java, *Stack* dapat dibuat menggunakan *array*, *ArrayList*, atau *class* bawaan seperti *Stack*. Pada praktikum ini, digunakan beberapa cara tersebut untuk menunjukkan bahwa meskipun berbeda implementasi, cara kerjanya tetap sama.

Operasi utama pada *Stack* adalah *push* (menambah data), *pop* (menghapus data teratas), dan *peek* (melihat data teratas). Selain itu, ada juga *isEmpty* untuk mengecek apakah *stack* kosong. Melalui praktikum ini, dapat dipahami bahwa *Stack* dapat digunakan untuk berbagai jenis data dan sangat berguna dalam pengolahan data secara berurutan.

2.2 Kode Pemograman

2.2.1 Class StackArray

```
2 package pekan3_2511531010;
3
4 public class stackArray_2511531010 {
5     static final int MAX = 1000;
6     int top;
7     int a[] = new int[MAX];
8
9     boolean isEmpty() {
10         return (top < 0);
11     }
12
13     stackArray_2511531010() {
14         top = -1;
15     }
16 }
```

```

17     boolean push(int x) {
18         if (top >= (MAX - 1)) {
19             System.out.println("Stack Overflow");
20             return false;
21         } else {
22             a[++top] = x;
23             System.out.println(x + " dimasukkan dalam stack");
24             return true;
25         }
26     }
27
28     int pop() {
29         if (top < 0) {
30             System.out.println("Stack Underflow");
31             return 0;
32         } else {
33             int x = a[top--];
34             return x;
35         }
36     }
37
38     int peek() {
39         if (top < 0) {
40             System.out.println("Stack Underflow");
41             return 0;
42         } else {
43             return a[top];
44         }
45     }
46
47     void print() {
48         for (int i = top; i > -1; i--) {
49             System.out.print(" " + a[i]);
50         }
51     }
52 }

```

Gambar 2.1

Class ini merupakan implementasi *Stack* menggunakan *array*. *Stack* memiliki kapasitas maksimum yang ditentukan. Variabel *top* digunakan untuk menunjukkan posisi elemen teratas. *Method* yang tersedia antara lain *push* untuk menambah data, *pop* untuk menghapus data, *peek* untuk melihat data teratas, dan *print* untuk menampilkan isi *stack*.

2.2.2 Class StackArray Driver

```

2     package pekan3_2511531010;
3

```



```

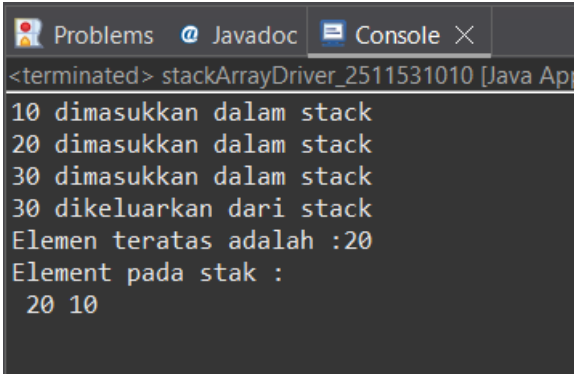
4 public class stackArrayDriver_2511531010 {
5
6     public static void main(String[] args) {
7         stackArray_2511531010 s= new stackArray_2511531010();
8         s.push(10);
9         s.push(20);
10        s.push(30);
11        System.out.println(s.pop() + " dikeluarkan dari stack");
12        System.out.println("Elemen teratas adalah :"+s.peek());
13        System.out.println("Element pada stak :");
14        s.print();
15    }
16 }
17 }

```

Gambar 2.2

Class ini berfungsi sebagai program utama untuk menguji *class* *stackArray*. Di dalamnya dibuat sebuah objek *stack*, kemudian dilakukan beberapa operasi seperti menambahkan data menggunakan *push*, menghapus data dengan *pop*, dan melihat elemen teratas dengan *peek*.

2.2.3 Output StackArray Driver



```

<terminated> stackArrayDriver_2511531010 [Java Applet]
10 dimasukkan dalam stack
20 dimasukkan dalam stack
30 dimasukkan dalam stack
30 dikeluarkan dari stack
Elemen teratas adalah :20
Element pada stak :
  20 10

```

Gambar 2.3

Output yang dihasilkan berupa tampilan proses penggunaan *stack*, seperti data yang dimasukkan ke dalam *stack*, data yang dikeluarkan, serta elemen yang berada di posisi teratas. Selain itu, ditampilkan juga isi *stack* setelah beberapa operasi dilakukan sehingga pengguna dapat melihat perubahan data.

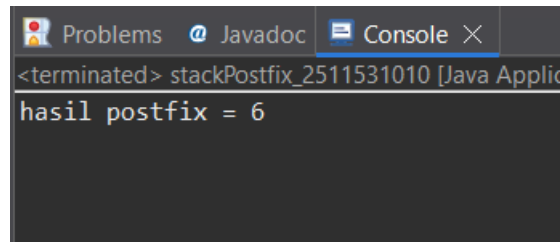
2.2.4 Class Stack Postfix

```
1 package pekan3_2511531010;
2
3 import java.util.Scanner;
4
5
6 public class stackPostfix_2511531010 {
7     public static int postfixEvaluate_2511531010(String expression) {
8         Stack<Integer> s = new Stack<Integer>();
9         Scanner input = new Scanner(expression);
10        while (input.hasNext()) {
11            if (input.hasNextInt()) { // operand
12                s.push(input.nextInt());
13            } else {
14                String operator_2511531010 = input.next();
15                int operand2_2511531010 = s.pop();
16                int operand1_2511531010 = s.pop();
17
18                if (operator_2511531010.equals("+")) {
19                    s.push(operand1_2511531010 + operand2_2511531010);
20                } else if (operator_2511531010.equals("-")) {
21                    s.push(operand1_2511531010 - operand2_2511531010);
22                } else if (operator_2511531010.equals("*")) {
23                    s.push(operand1_2511531010 * operand2_2511531010);
24                } else {
25                    s.push(operand1_2511531010 / operand2_2511531010);
26                }
27            }
28        }
29
30        input.close();
31        return s.pop();
32    }
33
34    public static void main(String[] args) {
35        System.out.println("hasil postfix = " + postfixEvaluate_2511531010("5 2 4 * + 7 -"));
36    }
37 }
```

Gambar 2.4

Class ini digunakan untuk menghitung nilai dari ekspresi *postfix* dengan memanfaatkan *Stack*. Program membaca *input* berupa *string* yang berisi angka dan operator, kemudian memprosesnya satu per satu. Jika yang dibaca adalah angka, maka akan dimasukkan ke dalam *stack*. Jika yang dibaca adalah operator, maka dua angka teratas akan diambil dari *stack* untuk dilakukan operasi perhitungan, lalu hasilnya dimasukkan kembali ke dalam *stack*. Proses ini dilakukan sampai seluruh ekspresi selesai diproses.

2.2.5 Output Stack Postfix



Gambar 2.5

Output dari program ini adalah hasil akhir dari perhitungan ekspresi *postfix*. Misalnya pada contoh yang diberikan, program akan menampilkan hasil perhitungan berdasarkan urutan operasi yang benar sesuai aturan *postfix*.

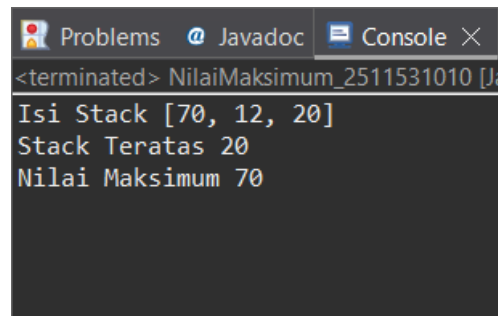
2.2.6 Class Nilai Maksimum

```
2 package pekan3_2511531010;
3
4 import java.util.Stack;
5
6 public class NilaiMaksimum_2511531010 {
7     public static int max(Stack<Integer> s) {
8         Stack<Integer> backup = new Stack<Integer>();
9         int maxValue = s.pop();
10        backup.push(maxValue);
11        while (!s.isEmpty()) {
12            int next = s.pop();
13            backup.push(next);
14            maxValue = Math.max(maxValue, next);
15        }
16        while (!backup.isEmpty()) {
17            s.push(backup.pop());
18        }
19        return maxValue;
20    }
21    public static void main(String[] args) {
22        Stack<Integer> s = new Stack<Integer>();
23        s.push(70);
24        s.push(12);
25        s.push(20);
26        System.out.println("Isi Stack "+s);
27        System.out.println("Stack Teratas "+s.peek());
28        System.out.println("Nilai Maksimum "+max(s));
29    }
30 }
```

Gambar 2.6

Class ini digunakan untuk mencari nilai maksimum yang terdapat dalam sebuah *stack*. Untuk menjaga agar data dalam *stack* tetap utuh, digunakan *stack* tambahan sebagai penyimpanan sementara. Selama proses pencarian, semua elemen dibandingkan untuk menemukan nilai terbesar, kemudian data dikembalikan ke *stack* semula.

2.2.7 Output Nilai Maksimum



```
<terminated> NilaiMaksimum_2511531010 [Ja
Isi Stack [70, 12, 20]
Stack Teratas 20
Nilai Maksimum 70
```

Gambar 2.7

Program akan menampilkan isi *stack* awal, elemen teratas, serta nilai maksimum yang ditemukan. Hal ini menunjukkan bahwa pencarian nilai maksimum dapat dilakukan tanpa merusak isi *stack*.

2.2.8 Class Siswa Stack

```
2 package pekan3_2511531010;
3
4 import java.util.ArrayList;
5
6 class Siswa_2511531010 {
7     String nama;
8     int nim;
9
10    public Siswa_2511531010(String nama, int nim) {
11        this.nama = nama;
12        this.nim = nim;
13    }
14
15    @Override
16    public String toString() {
17        return "NIM: " + nim + ", Nama: " + nama;
18    }
19 }
20
21 public class SiswaStack_2511531010 {
22     private ArrayList<Siswa_2511531010> stack;
```

```

23
24     public SiswaStack_2511531010() {
25         stack = new ArrayList<>();
26     }
27
28     public void push_2511531010(Siswa_2511531010 mhs) {
29         stack.add(mhs);
30     }
31
32     public Siswa_2511531010 pop_2511531010() {
33         if (!isEmpty_2511531010()) {
34             return stack.remove(stack.size() - 1);
35         }
36         return null;
37     }
38
39     public Siswa_2511531010 peek_2511531010() {
40         if (!isEmpty_2511531010()) {
41             return stack.get(stack.size() - 1);
42         }
43         return null;
44     }
45
46     public boolean isEmpty_2511531010() {
47         return stack.isEmpty();
48     }
49
50     public void tampilkanSiswa_2511531010() {
51         for (int i = stack.size() - 1; i >= 0; i--) {
52             System.out.println(stack.get(i));
53         }
54     }
55
56     public static void main(String[] args) {
57         SiswaStack_2511531010 studentStack_2511531010 = new
SiswaStack_2511531010();
58
59         Siswa_2511531010 mhs1 = new Siswa_2511531010("Ali", 1);
60         Siswa_2511531010 mhs2 = new Siswa_2511531010("Boby", 2);
61         Siswa_2511531010 mhs3 = new Siswa_2511531010("Charles", 3);
62
63         studentStack_2511531010.push_2511531010(mhs1);
64         studentStack_2511531010.push_2511531010(mhs2);
65         studentStack_2511531010.push_2511531010(mhs3);
66
67         System.out.println("Siswa di dalam stack:");
68         studentStack_2511531010.tampilkanSiswa_2511531010();
69
70         System.out.println("Siswa teratas: " +
studentStack_2511531010.peek_2511531010());
71         System.out.println("Mengeluarkan siswa teratas dari stack:
" + studentStack_2511531010.pop_2511531010());
72         System.out.println("Daftar siswa setelah di pop:");
73         studentStack_2511531010.tampilkanSiswa_2511531010();

```

```

74     }
75 }

```

Gambar 2.8

Terdapat *class* Siswa sebagai data yang berisi nama dan NIM. *Class* ini menyediakan *method* seperti *push* untuk menambah siswa, *pop* untuk menghapus siswa teratas, *peek* untuk melihat siswa teratas, serta *method* untuk menampilkan seluruh data.

2.2.9 Output Siswa Stack

```

<terminated> SiswaStack_2511531010 [Java Application] C:\Users\ASUS\p2\pool\pl
Siswa di dalam stack:
NIM: 3, Nama: Charles
NIM: 2, Nama: Bobby
NIM: 1, Nama: Ali
Siswa teratas: NIM: 3, Nama: Charles
Mengeluarkan siswa teratas dari stack: NIM: 3, Nama: Charles
Daftar siswa setelah di pop:
NIM: 2, Nama: Bobby
NIM: 1, Nama: Ali

```

Gambar 2.9

Output yang dihasilkan berupa daftar siswa dalam *stack*, dimulai dari data yang terakhir dimasukkan. Program juga menampilkan siswa teratas, data yang dikeluarkan saat *pop*, serta kondisi *stack* setelah data dihapus.

2.2.10 Class Latihan Stack

```

2  package pekan3_2511531010;
3
4  import java.util.Stack;
5  public class latihanStack_2511531010 {
6
7      public static void main(String[] args) {
8          Stack<Integer> s = new Stack<Integer>();
9          s.push(42);
10         s.push(-3);
11         s.push(17);
12         System.out.println("nilai stack = "+s);
13         System.out.println("nilai pop = "+s.pop());
14         System.out.println("nilai stack setelah pop = "+s);
15         System.out.println("nilai peek"+s.peek());
16         System.out.println("nilai stack setelah peek= "+s);
17     }

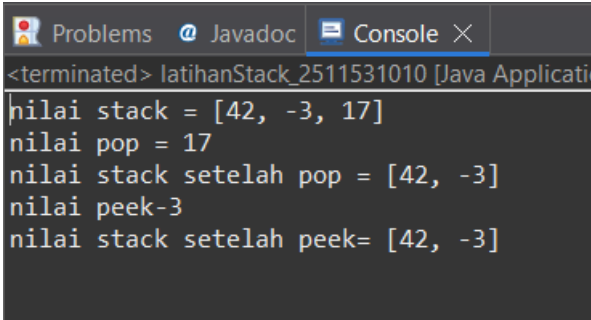
```

```
18  
19 }
```

Gambar 2.10

Program menggunakan *class Stack* untuk menyimpan data bertipe *integer*. Beberapa operasi dasar seperti *push*, *pop*, dan *peek* dilakukan untuk menunjukkan cara kerja *stack* secara langsung.

2.2.11 Output Latihan Stack



```
<terminated> latihanStack_2511531010 [Java Applicati  
nilai stack = [42, -3, 17]  
nilai pop = 17  
nilai stack setelah pop = [42, -3]  
nilai peek=3  
nilai stack setelah peek= [42, -3]
```

Gambar 2.11

Program akan menampilkan isi *stack* setelah data dimasukkan, hasil dari operasi *pop*, serta isi *stack* setelah data dihapus. Selain itu, ditampilkan juga elemen teratas menggunakan *peek*, sehingga terlihat bagaimana perubahan data dalam *stack* terjadi.

BAB III

PENUTUP

3.1 Kesimpulan

Dari praktikum yang telah dilakukan, dapat disimpulkan bahwa *Stack* merupakan salah satu struktur data yang bekerja dengan prinsip *Last In First Out* (LIFO), yaitu data yang terakhir dimasukkan akan menjadi data pertama yang dikeluarkan. Melalui beberapa implementasi yang telah dibuat, dapat dipahami bahwa *stack* memiliki fleksibilitas dalam penerapannya. Meskipun cara implementasinya berbeda, operasi dasar seperti *push*, *pop*, dan *peek* tetap menjadi inti dari penggunaan *Stack*.

Selain itu, praktikum ini juga menunjukkan bahwa *Stack* tidak hanya digunakan untuk data sederhana seperti angka, tetapi juga dapat digunakan untuk objek, seperti data siswa. Bahkan, *Stack* juga dapat dimanfaatkan untuk menyelesaikan masalah yang lebih kompleks, seperti evaluasi ekspresi *postfix* dan pencarian nilai maksimum dalam suatu kumpulan data.

DAFTAR PUSTAKA

- [1] Oracle Corporation. 2023. *Class Stack (Java Platform SE)*.
Diakses dari <https://docs.oracle.com/javase/8/docs/api/java/util/Stack.html>.

TUGAS PRAKTIKUM

Soal:

2. Aturan Penamaan (Wajib Dipatuhi)

- 1) Nama Kelas : Gunakan akhiran NIM lengkap.
Contoh : Website_2311532008 dan Browser_2311532008.
- 2) Nama Variabel Atribut : Setiap variabel atribut di kelas ADT gunakan akhiran 4 digit terakhir NIM.
Contoh (NIM ...2008): String judul_2008, String url_2008.

3. Struktur Program

- 1) Kelas ADT Website (Website_NIMLengkap)
Atribut :
 - Judul Website
 - URL WebsiteMethod :
 - Constructor untuk menginisialisasi atribut.
 - Selektor (Getter) untuk setiap atribut.
 - Mutator (Setter) untuk setiap atribut.
- 2) Kelas Driver (Browser_NIMLengkap)
 - Kunjungi Website (Push) : Memasukkan data website baru yang sedang dikunjungi ke dalam tumpukan.
 - Tombol Back (Pop) : Menghapus halaman website terakhir (paling atas) dan menampilkan judul halaman yang berhasil dihapus.
 - Lihat Halaman Saat Ini (Peek) : Menampilkan detail website yang sedang aktif dibuka tanpa menghapusnya dari tumpukan.
 - Cek Status History : Menampilkan jumlah total riwayat yang tersimpan atau mengecek apakah riwayat kosong menggunakan .isEmpty().

Kode pemograman:

1. Class Website_2511531010

```
2. package pekan3_2511531010;
3.
4. public class Website_2511531010 {
5.
6.     private String judul_1010;
7.     private String url_1010;
8.
9.     // constructor
10.    public Website_2511531010(String judul, String url) {
11.        this.judul_1010 = judul;
12.        this.url_1010 = url;
13.    }
14.
15.    // getter
16.    public String getJudul_1010() {
17.        return judul_1010;
18.    }
19.
20.    public String getUrl_1010() {
21.        return url_1010;
22.    }
23.
24.    // setter
25.    public void setJudul_1010(String judul) {
26.        this.judul_1010 = judul;
27.    }
28.
29.    public void setUrl_1010(String url) {
30.        this.url_1010 = url;
31.    }
32.
33.    @Override
34.    public String toString() {
35.        return "Judul: " + judul_1010 + ", URL: " + url_1010;
36.    }
37. }
```

2. Class Browser_2511531010

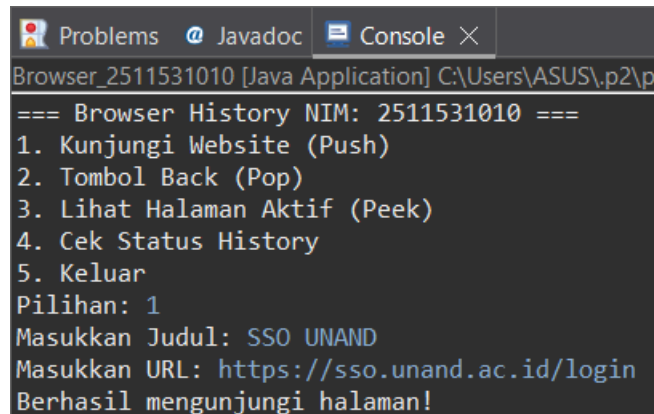
```
1. package pekan3_2511531010;
2.
3. import java.util.Scanner;
4. import java.util.Stack;
5.
6. public class Browser_2511531010 {
7.
8.     public static void main(String[] args) {
9.         Stack<Website_2511531010> history = new Stack<>();
10.        Scanner sc = new Scanner(System.in);
11.        int pilihan;
12.
13.        do {
14.            System.out.println("=== Browser History NIM:
2511531010 ===");
15.            System.out.println("1. Kunjungi Website (Push)");
16.            System.out.println("2. Tombol Back (Pop)");
17.            System.out.println("3. Lihat Halaman Aktif (Peek)");
18.            System.out.println("4. Cek Status History");
19.            System.out.println("5. Keluar");
20.            System.out.print("Pilihan: ");
21.            pilihan = sc.nextInt();
22.            sc.nextLine();
23.
24.            switch (pilihan) {
25.                case 1:
26.                    System.out.print("Masukkan Judul: ");
27.                    String judul = sc.nextLine();
28.
29.                    System.out.print("Masukkan URL: ");
30.                    String url = sc.nextLine();
31.
32.                    history.push(new Website_2511531010(judul,
url));
33.                    System.out.println("Berhasil mengunjungi
halaman!");
34.                    break;
35.
36.                case 2:
37.                    if (history.isEmpty()) {
38.                        System.out.println("History kosong, tidak
bisa kembali.");
39.                    } else {
40.                        Website_2511531010 removed =
history.pop();
41.                        System.out.println("Kembali dari: " +
removed.getJudul_1010());
42.                    }
43.                    break;
44.
45.                case 3:
46.                    if (history.isEmpty()) {
```

```

47.         System.out.println("Tidak ada halaman
aktif.");
48.     } else {
49.         System.out.println("Halaman aktif.");
50.         System.out.println(history.peek());
51.     }
52.     break;
53.
54.     case 4:
55.         if (history.isEmpty()) {
56.             System.out.println("History kosong.");
57.         } else {
58.             System.out.println("Jumlah history: " +
history.size());
59.         }
60.         break;
61.
62.     case 5:
63.         System.out.println("Terima kasih!");
64.         break;
65.
66.     default:
67.         System.out.println("Pilihan tidak valid.");
68.     }
69.
70. } while (pilihan != 5);
71.
72. sc.close();
73. }
74. }

```

3. Output



```

Problems  @ Javadoc  Console X
Browser_2511531010 [Java Application] C:\Users\ASUS\p2\p...
=== Browser History NIM: 2511531010 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. Keluar
Pilihan: 1
Masukkan Judul: SSO UNAND
Masukkan URL: https://sso.unand.ac.id/login
Berhasil mengunjungi halaman!

```

Program terdiri dari dua bagian utama, yaitu *class* ADT (*Website_2511531010*) dan *class driver* (*Browser_2511531010*). *Class Website* digunakan untuk menyimpan data *website* yang terdiri

dari judul dan URL. Class ini dilengkapi dengan *constructor*, *getter*, dan *setter* sehingga data dapat diakses dan diubah dengan mudah.

Sedangkan *class Browser* berfungsi sebagai program utama yang mengelola data menggunakan struktur *Stack*. Di dalamnya terdapat menu interaktif yang memungkinkan pengguna untuk melakukan beberapa operasi, yaitu mengunjungi *website* (*push*), kembali ke halaman sebelumnya (*pop*), melihat halaman yang sedang aktif (*peek*), serta mengecek jumlah atau kondisi *history*.

Penggunaan *Stack* dalam program ini sangat sesuai karena mengikuti konsep *Last In First Out*, di mana halaman terakhir yang dikunjungi akan diproses terlebih dahulu saat pengguna kembali ke halaman sebelumnya. Selain itu, program juga dilengkapi dengan pengecekan kondisi *stack* kosong agar tidak terjadi *error* saat melakukan operasi *pop* atau *peek*.